CUDAnet · COMPANION DOCUMENT 1 OF 2

The CUDAnet Codebase, Explained

A line-by-line technical walkthrough of `mnist_nn_cuda.cu` — what every kernel does, how the three training-loop code paths differ, and how the Two-Phase Gradient Accumulation fix described in the paper actually appears in code.

Reference for: CUDAnet — A Micro-Footprint, Dependency-Free Deep Learning Engine

Prepared for Samartha H V | Manipal Institute of Technology Bengaluru

Contents

1. **1** How This Document Fits Together
2. **2** Repository Layout
3. **3** Build System — Makefile Targets Mapped to Paper Configs
4. **4** Runtime CLI & the `network_structure` Layout
5. **5** Compile-Time Feature Flags Reference
6. **6** The Three Training-Loop Code Paths
7. **7** Two-Phase Gradient Accumulation — In Code
8. **8** Batch Normalization Kernel Walkthrough
9. **9** Optimizer Kernels (SGD / Momentum / Adam)
10. **10** Evaluation Kernel & the Stack-Overflow Fix
11. **11** Known Issues, Dead Code & Gotchas
12. **12** Reproducing the Paper's Numbers
13. **13** Glossary

1. How This Document Fits Together

You have three artifacts now: the **paper** (the claims and the evidence), the **presentation deck** (the 10-minute story), and the **codebase** (the ground truth everything else is describing). This document is a guided tour of the codebase — specifically `src/mnist_nn_cuda.cu`, a single ~2,300-line CUDA C file that is the entire training engine. Everything here was verified by reading the actual source, not reconstructed from memory, so line numbers and function names are exact as of this writing.

The companion document, *"The CUDAnet Paper, Explained,"* covers the same material from the claims-and-evidence side — read that one for the "why," this one for the "how."

> **How to use this document**
>
> Each code excerpt below is taken verbatim from `src/mnist_nn_cuda.cu` with its starting line number tagged, so you can jump straight to the source when presenting or defending your work.

2. Repository Layout

Path	Purpose
`src/mnist_nn_cuda.cu`	The entire CUDA training/inference engine (~2,300 lines). One file, compiled multiple times with different `-D` flags to produce each optimizer/BN/WMMA variant.
`src/mnist_nn_serial_c.c`	Single-threaded (and OpenMP multi-threaded via `serial_omp`) CPU baseline used for the speedup comparison.
`src/load_data.h`	MNIST/Fashion-MNIST IDX-format loader; reads the raw ubyte files into host arrays, called once at startup by `load_mnist()`.
`src/infer.cu`	Standalone inference-only binary — loads a saved checkpoint and classifies a single image (used for the <0.4 ms Jetson/edge latency figure).
`Makefile`	Defines every build target (see §3) — this is effectively the experiment configuration file.
`data/`	Raw MNIST IDX files.
`results/*.txt`	Captured stdout from each training run — the raw data behind every figure/table.
`scripts/visualize_advanced.py`, `visualize_training.py`	Matplotlib scripts that parse `results/*.txt` and produce `figures/*.png`.
`main.tex`	The IEEE conference paper source.

3. Build System — Makefile Targets Mapped to Paper Configs

There is no runtime flag to switch optimizer or enable BN — these are **compile-time** choices via preprocessor macros (`-DOPTIMIZER_TYPE=…`, `-DUSE_BATCH_NORM=1`, etc.), each producing a separate binary. This is why the Makefile has one target per experiment configuration rather than one binary with runtime flags for everything.

Make target	Flags added to `-arch sm_86 -O2`	Paper configuration
`cuda`	(none — defaults)	SGD
`cuda_momentum`	`-DOPTIMIZER_TYPE=1`	Momentum
`cuda_adam`	`-DOPTIMIZER_TYPE=2`	Adam
`cuda_adam_lr`	`-DOPTIMIZER_TYPE=2 -DUSE_LR_SCHEDULE=1 -DLR_SCHEDULE_TYPE=2`	Adam + LR (cosine)
`cuda_batchnorm`	`-DUSE_BATCH_NORM=1`	BN + SGD
`cuda_full`	`-DOPTIMIZER_TYPE=2 -DUSE_LR_SCHEDULE=1 -DUSE_BATCH_NORM=1 -DLR_SCHEDULE_TYPE=2`	BN + Adam + LR
`cuda_wmma`	`-DUSE_TENSOR_CORES=1 -DOPTIMIZER_TYPE=2 -DUSE_LR_SCHEDULE=1`	*Experimental* — not in reported paper results
`cuda_nvml` / `cuda_nvml_full`	adds `-DUSE_NVML -lnvidia-ml`	Same as Adam+LR / BN+Adam+LR, with NVML power sampling compiled in
`cuda_fashion` / `cuda_fashion_full`	adds `-DFASHION_MNIST=1`	Fashion-MNIST variants (not in the paper)
`serial` / `serial_omp`	gcc, `-O2` (+ `-fopenmp`)	CPU baseline(s)

> **Note**
> `cuda_wmma` exists and compiles, but it is not one of the six configurations reported in the paper's tables — Tensor Cores are listed under *Future Work* in the paper even though an experimental kernel path already exists in the code (see §6.2 and §11).

4. Runtime CLI & the `network_structure` Layout

Whatever binary you build, it takes eight positional arguments (`main()`, line 1676):

```
./cuda <nl> <nh> <ne> <nb> <alpha> <output_type> <log_interval> <save_flag>

nl            number of hidden layers (must be < 21)
nh            neurons per hidden layer
ne            number of epochs
nb            batch size
alpha         learning rate
output_type   0 = Sigmoid + MSE   |   1 = Softmax + Cross-Entropy
log_interval  print progress every N epochs (optional, default 1)
save_flag     0 = none, 1 = save checkpoint after training, 2 = load + continue training (optional)
```

Example used throughout the paper's experiments: `./cuda_adam 2 128 20 2048 0.01 1 5 0` — 2 hidden layers × 128 neurons, 20 epochs, batch 2048, LR 0.01, softmax+CE, log every 5 epochs.

Flat parameter layout

All weights and biases for every layer live in **one contiguous array each** (not per-layer arrays), addressed via precomputed offset tables in a single `network_structure` struct (line 106):

```
struct network_structure {
    int layers[22];        // neuron count per layer, layers[0]=784 (input), layers[L-1]=10 (output)
    int weights_pstn[22];  // flat-array offset where layer l's weight block begins
    int biases_pstn[22];   // flat-array offset where layer l's bias block begins
    int dza_pstn[22];      // offset into the per-sample activation/delta/z buffers
    int num_nodes;         // total neurons across all layers (= activation buffer width per sample)
    int L;                 // number of layers including input+output (= nl + 2)
#if USE_BATCH_NORM
    int bn_gamma_pstn[22]; int bn_beta_pstn[22]; int num_bn_params;
#endif
};
```

This is why the paper calls addressing **O(1)**: no pointer chasing between layers, just `weights[ns.weights_pstn[l] + i + j*ns.layers[l+1]]` — a single array index computed from precomputed offsets, which is friendly to the GPU's memory coalescing and cache behaviour.

5. Compile-Time Feature Flags Reference

Everything below is a `#define` near the top of the file (lines 26–104). Most are **disabled by default** and were experimental explorations that did not make it into the paper's headline results — worth knowing so you don't accidentally claim a feature is "in the paper" when it's really dead-default code sitting in the file.

Flag	Default	Used in paper's 6 reported configs?
`OPTIMIZER_TYPE`	0 (SGD)	Yes — the primary axis of comparison
`USE_BATCH_NORM`	0 (off)	Yes — BN+SGD, BN+Adam+LR
`USE_LR_SCHEDULE` / `LR_SCHEDULE_TYPE`	0 / 1 (exp. decay)	Yes, but paper only uses `LR_SCHEDULE_TYPE=2` (cosine)
`USE_SHARED_MEMORY`	1 (on)	Yes — always on; tiled 32×32 shared-memory matmul is the baseline, not optional
`USE_VALIDATION_SPLIT` / `VALIDATION_RATIO`	1 / 0.2	Yes — the 48k/12k split reported in §VI-B of the paper
`USE_TENSOR_CORES`	0 (off)	**No** — experimental WMMA path, only built via `cuda_wmma`, SGD-only, listed as Future Work in the paper
`USE_MIXED_PRECISION`	0 (off)	**No** — comment in the code notes it's "~1% slower" without a true WMMA kernel behind it
`USE_DROPOUT` / `DROPOUT_RATE`	0 (off)	**No** — comment notes "+13.6% overhead, +0.04% test acc" i.e. tried and not worth it
`USE_ASYNC_STREAMS`	0 (off)	**No** — comment notes no benefit since all data is already resident on GPU
`USE_NVML`	undefined	Only for the energy-sampling build variants, not the core 6 configs
`FASHION_MNIST`	undefined	**No** — alternate dataset, not reported in the paper

> **Why this matters for Q&A**
>
> If someone asks "did you try mixed precision / Tensor Cores / dropout?" — the honest, precise answer is *yes, the code has all three behind compile flags, but none were used for the reported numbers*: dropout because the accuracy gain was negligible for the overhead, mixed precision because it needs a real WMMA kernel to pay off, and Tensor Cores because that WMMA kernel exists but is experimental/SGD-only and is explicitly future work.

6. The Three Training-Loop Code Paths

Inside the per-batch loop (`main()`, starting line 2004) there are exactly three mutually-exclusive code paths, selected at **compile time** by nested `#if / #elif / #else` (line 2022 onward). Only one of these is compiled into any given binary.

6.1 `#if USE_BATCH_NORM` — the staged per-layer kernel path

This is the architecture described in the paper's Fig. 1 and §III-B. Used by `cuda_batchnorm` and `cuda_full`. The forward pass is split into one kernel launch per layer with an explicit `cudaDeviceSynchronize()` barrier between the linear transform and the BN statistics kernel:

```
line 2042
init_activations_kernel<<<nblocks, nthreads>>>(train_image_d, a_list_d, batch_d, nb, ns);

for (int l = 1; l < ns.L; l++) {
    fwd_linear_kernel<<<nblocks, nthreads>>>(weights_d, biases_d, a_list_d, z_list_d, l, ns, nb);
    if (l < ns.L-1) {
        cudaDeviceSynchronize();              // all z[l] written before stats
        bn_stats_kernel<<<ns.layers[l], 256>>>(z_list_d, ...);   // one block per neuron
        bn_apply_kernel<<<nblocks, nthreads>>>(z_list_d, ...);
    }
    apply_activation_kernel<<<nblocks, nthreads>>>(a_list_d, z_list_d, l, ns, nb, output_type);
}
```

The backward pass mirrors this exactly, with `bn_backward_stats_kernel` / `bn_backward_delta_kernel` pairs separated by another sync (line 2069–2090). This is the only place in the codebase where BN statistics are computed correctly — see §11 for a subtlety about a second, unreachable BN implementation.

6.2 `#elif USE_TENSOR_CORES` — experimental WMMA path

Built only via `cuda_wmma`. Uses warp-level `wmma::mma_sync` on 16×16×16 FP16 tiles (`wmma_fwd_kernel`, line 988; `wmma_bwd_delta_kernel`, line 1056), falling back to the scalar `fwd_linear_kernel` / `backward_delta_kernel` for any layer whose dimensions aren't divisible by 16 (line 2148, 2173). This path is **SGD-only** — it calls `sgd_layer_update_kernel` directly (line 2195) with no gradient-accumulation/2PGA logic, because it predates the optimizer-state race-condition fix and was never re-integrated with Momentum/Adam. It is not one of the paper's reported configurations.

6.3 `#else` — the monolithic `one_learning_cycle` kernel

Used by every non-BN configuration: SGD, Momentum, Adam, Adam+LR. This is **one single kernel launch per batch** — each thread does the entire forward pass, backward pass, and gradient accumulation for one sample, with no intermediate synchronization needed because (without BN) no thread ever needs to read another thread's intermediate value:

```
line 2203
one_learning_cycle<<<nblocks, nthreads>>>(train_image_d, train_label_d, weights_d, biases_d,
    deltas_d, a_list_d, z_list_d, batch_d, output_type, ns, nb, alpha_nb, epoch
    /* + grad_w_d, grad_b_d, velocity/moment buffers depending on OPTIMIZER_TYPE */);

#if OPTIMIZER_TYPE == 1 || OPTIMIZER_TYPE == 2
    cudaDeviceSynchronize();                        // Phase 1 → Phase 2 barrier
    apply_optimizer_update<<<opt_nblocks, opt_nthreads>>>(weights_d, biases_d,
        grad_w_d, grad_b_d, num_weights, num_biases, current_lr, nb, epoch + 1, ...);
#endif
```

This is the fastest path — one kernel launch does forward+backward+gradient-accumulation for the whole batch — which is why the paper's fastest configs (plain SGD at 0.94 s/epoch) are all in this branch.

Code path	Trigger	Kernel launches / batch	Paper configs using it
Staged BN path	`USE_BATCH_NORM=1`	~4×(L−1) small kernels + syncs	BN+SGD, BN+Adam+LR
WMMA path	`USE_TENSOR_CORES=1`	~2×L kernels + syncs	*none* (experimental only)
Monolithic `one_learning_cycle`	default	1 (+1 for Momentum/Adam Phase 2)	SGD, Momentum, Adam, Adam+LR

7. Two-Phase Gradient Accumulation — In Code

The paper's central contribution (2PGA) is not a separate module — it's a pattern applied in two different places depending on which of the three paths above is active. In both cases the shape is identical: **Phase 1** accumulates gradients via `atomicAdd` into a shared buffer that no thread reads from during Phase 1, then a barrier, then **Phase 2** is a completely separate kernel where each parameter has exactly one owning thread.

7.1 Phase 1 — accumulate only, never read-modify-write optimizer state

`line 910 — grad_accum_layer_kernel (BN path)`

```
__global__ void grad_accum_layer_kernel(float* grad_w, float* grad_b, float* deltas,
 float* a_list, int l, struct network_structure ns, int nb)
{
    int id = threadIdx.x + blockDim.x * blockIdx.x;
    if (id >= nb) return;
    ...
    atomicAdd(&grad_w[...], d[...] * a[...]);   // gradient only — NOT weights, NOT velocity/moments
    atomicAdd(&grad_b[...], d[...]);
}
```

`line 1614 — inside one_learning_cycle (monolithic path)`

```
#if OPTIMIZER_TYPE == 0                    // SGD — no state, so update directly, same kernel
    atomicAdd(&biases[bias_idx], (-alpha_nb) * local_grad_b);
#elif OPTIMIZER_TYPE == 1 || OPTIMIZER_TYPE == 2   // Momentum/Adam — accumulate only, defer the update
    atomicAdd(&grad_b[bias_idx], local_grad_b);
#endif
```

Notice the branch on `OPTIMIZER_TYPE`: SGD is stateless, so it's safe (and faster) to apply the update *inside* Phase 1 directly — that's the pre-2PGA behaviour, kept as-is because it was never broken. Momentum and Adam take the accumulate-only branch instead.

7.2 Phase 2 — one thread per parameter, single kernel, race-free by construction

`line 1414 — apply_optimizer_update`

```
__global__ void apply_optimizer_update(float* weights, float* biases,
 float* grad_w, float* grad_b, int num_weights, int num_biases, float lr, int nb, int epoch, ...)
{
    int idx = threadIdx.x + blockIdx.x * blockDim.x;   // idx == a specific weight, exactly once
#if OPTIMIZER_TYPE == 2   // Adam
    if (idx < num_weights){
        float grad = grad_w[idx] / (float)nb;          // average over the batch
        m_w[idx] = ADAM_BETA1*m_w[idx] + (1-ADAM_BETA1)*grad;
        v_w[idx] = ADAM_BETA2*v_w[idx] + (1-ADAM_BETA2)*grad*grad;
        float m_hat = m_w[idx] / (1 - powf(ADAM_BETA1, epoch));
        float v_hat = v_w[idx] / (1 - powf(ADAM_BETA2, epoch));
        weights[idx] -= lr * m_hat / (sqrtf(v_hat) + ADAM_EPSILON);
        grad_w[idx] = 0.0f;                              // reset accumulator for next batch
    }
#endif
}
```

There is no `atomicAdd` anywhere in this kernel — thread `idx` is the sole owner of weight `idx` for the entire kernel lifetime, so a plain read-modify-write is safe. This is why the paper can claim Phase 2 needs "no hardware atomics, no locking": the race is eliminated by construction, not by synchronization primitives layered on top of the old naive kernel.

> **The one-line mental model**
>
> Phase 1 = "many threads, one gradient buffer, write-only." Phase 2 = "many threads, many buffers, but each thread touches exactly one element of each." A race requires two threads writing the *same* address without ordering — Phase 2 never has two threads targeting the same address, so the race is structurally impossible, not just unlikely.

8. Batch Normalization Kernel Walkthrough

Six kernels implement BN end-to-end, matching equations (2)–(3) in the paper:

Kernel	Line	Role
`fwd_linear_kernel`	677	z = Wa + b for one layer, all samples in parallel
`bn_stats_kernel`	695	One block *per neuron*, 256 threads reduce over the batch to compute μ_j, σ^2_j — this is the one place per-neuron statistics genuinely require every sample's z to already be written, hence it must run *after* the sync following `fwd_linear_kernel`
`bn_apply_kernel`	734	Normalizes z using the stats just computed, applies learnable γ, β
`apply_activation_kernel`	751	ReLU (hidden) or softmax (output)
`bn_backward_stats_kernel`	807	Backward reduction: computes $\sum \partial y$ and $\sum \partial y \cdot \hat{z}$ needed for the BN input gradient, plus accumulates `dgamma` / `dbeta`
`bn_backward_delta_kernel`	864	Applies the BN backward formula to propagate the gradient through the normalization

BN's own learnable parameters (γ, β) are updated by a seventh, optimizer-aware kernel, `bn_params_update_kernel` (line 930) — it has its own SGD/Momentum/Adam branches, entirely separate from the main weight/bias optimizer kernel, and it zeroes its own gradient accumulators (`dgamma` / `dbeta`) at the end of every call.

9. Optimizer Kernels (SGD / Momentum / Adam)

Three update rules, each gated by `#if OPTIMIZER_TYPE == N`, appear (with minor variable-naming differences) in *three* places: `apply_optimizer_update` (main weights/biases, non-BN or BN path), `bn_params_update_kernel` (BN's own γ/β), and inline in `sgd_layer_update_kernel` /the SGD branch of `one_learning_cycle` for the stateless case.

Optimizer	`OPTIMIZER_TYPE`	Update rule (as implemented)
SGD	0	`w -= lr * grad`, applied directly inside Phase 1, no separate Phase 2 kernel needed
Momentum	1	`v = 0.9*v + grad; w -= lr*v` — note: implemented as biased-toward-recent (no `(1-β)` term on the gradient itself, matching Eq. 7–8 in the paper)
Adam	2	`m,v` updated with `ADAM_BETA1=0.9`, `ADAM_BETA2=0.999`; bias-corrected via `powf(BETA, epoch)`; `w -= lr * m_hat / (sqrt(v_hat) + 1e-8)`

All three read `grad_w[idx] / (float)nb` first — the batch-mean gradient — before applying the update, matching Eq. 6 in the paper (`G_W / B`).

10. Evaluation Kernel & the Stack-Overflow Fix

`evaluate_kernel` (line 1181) is what the paper's §V-C describes. The current code already has the fix applied — activation/pre-activation buffers are passed in as pointers into pre-allocated global VRAM, *not* declared as local stack arrays:

```
line 1193
// Use global VRAM buffers - avoids register/stack blowout on large images
float* z = &global_z[id * ns.num_nodes];
float* a = &global_a[id * ns.num_nodes];
```

The bug this fixed: with 10,000 test images and a naive `float z[1024]; float a[1024];` declared per-thread on the stack, that's 10,000 × 1024 × 4 bytes ≈ 80 MB of local/register memory requested simultaneously — enough to crash the CUDA context. Note that `wxa[IMAGE_SIZE]` (a 784-float array, line 1205) is *still* stack-allocated per thread inside the layer loop — this works because it's reused per-layer rather than accumulated per-image, but it's worth knowing this buffer exists if register pressure ever becomes a bottleneck for larger images.

11. Known Issues, Dead Code & Gotchas

log_interval = 0 causes SIGFPE

Line 1991: `if (epoch % log_interval == 0)`. If `log_interval` is passed as `0` (the 7th CLI argument), this is an integer division by zero and the process crashes with SIGFPE. Always pass a positive value (1 = log every epoch, 5 = every 5th, etc.).

Unreachable BN branch inside `one_learning_cycle`

`one_learning_cycle` (line 1463) contains its own `#if USE_BATCH_NORM` branch (lines ~1500–1540 forward, ~1591–1671 backward) that computes BN inline via `batch_norm_forward_sample` using per-thread local arrays `bn_mean_all[10][256]`. This code is **never actually executed** in any built binary: when `USE_BATCH_NORM=1`, `main()`'s outer `#if USE_BATCH_NORM` branch (§6.1) takes over the whole per-batch loop and never calls `one_learning_cycle` at all (see line 2034 vs. line 2203 — mutually exclusive `#if/#elif/#else`). It reads as an earlier, in-kernel prototype of BN support that was superseded by the correct staged-kernel design but never deleted. Harmless (it doesn't compile into the non-BN binaries either, since the `#if` is false there too), but worth knowing so you don't describe it as "the" BN implementation if asked to walk through the code live — the staged kernels in §8 are the ones that actually run.

avgTime off-by-one

The reported average time-per-epoch divides accumulated time by `(epoch-1)` rather than `epoch` at the point it's printed, which shows `inf` at epoch 1 and is systematically overestimated by roughly one epoch's worth out of the total — about 5% for a 19-datapoint run. Doesn't affect the underlying CUDA-event timings used for the paper's Table I (those come from `cudaEventElapsedTime` directly), only the human-readable "average" line printed to stdout during training.

WMMA / mixed-precision / dropout are compiled but cold

As covered in §5 — these exist as complete, compilable code paths (not stubs), but are switched off by default and were not used to produce any of the paper's six reported configurations.

12. Reproducing the Paper's Numbers

Paper config	Build	Run
SGD	`make cuda`	`./cuda 2 128 20 2048 0.1 1 5 0`
Momentum	`make cuda_momentum`	`./cuda_momentum 2 128 20 2048 0.1 1 5 0`
Adam	`make cuda_adam`	`./cuda_adam 2 128 20 2048 0.01 1 5 0`
Adam + LR (cosine)	`make cuda_adam_lr`	`./cuda_adam_lr 2 128 20 2048 0.01 1 5 0`
BN + SGD	`make cuda_batchnorm`	`./cuda_batchnorm 2 128 20 2048 0.1 1 5 0`
BN + Adam + LR	`make cuda_full`	`./cuda_full 2 128 20 2048 0.01 1 5 0`

All six use the same architecture (`nl=2, nh=128`), 20 epochs, batch 2048, on the standard MNIST split described in the paper's §VI-B.

13. Glossary

Term	Meaning in this codebase
`ns` / `network_structure`	The struct holding every layer-size and offset table (§4) — passed by value into most kernels
`a_list` / `z_list` / `deltas`	Per-sample, per-neuron buffers: post-activation, pre-activation, and backprop error respectively — each sized `nb × ns.num_nodes`
`grad_w` / `grad_b`	Phase-1 gradient accumulation buffers (Momentum/Adam only) — zeroed every batch, summed via `atomicAdd`, consumed and re-zeroed by Phase 2
`alpha_nb`	`learning_rate / batch_size` — pre-divided so the SGD direct-update kernel doesn't need to divide per-thread
`dza_pstn`	"delta/z/activation position" — the offset table shared by `a_list`, `z_list`, and `deltas` (they're indexed identically per-sample)
2PGA	Two-Phase Gradient Accumulation — the paper's name for the Phase-1-accumulate / Phase-2-apply pattern described in §7